Cross-Platform Release Plan

Targets (priority order): Android, iOS, Steam (Windows / macOS / Linux).

Source codebase: Phaser 4 (WebGL) + Preact + Vite, shipping today as a PWA on GitHub Pages. The plan below keeps that codebase as the single source of truth and wraps it for each platform — rewriting in a native engine (Unity / Godot) is off the table.

---

1. Landscape (2026)

Significant shifts since the last time we looked:

- **Tauri 2.0 is GA.** Released October 2024, with mobile support (iOS + Android) part of the stable release — not alpha/beta. Production apps built on it now include Hoppscotch, Spacedrive, Padloc, and AppFlowy. Active security audits. Roughly 96% smaller binaries than Electron (~5–15 MB vs ~100 MB).
- **Electron 34+** tightened the security model and remains the default for desktop + Steam.
- **Capacitor** remains the mature mobile default — 7 years in production, Ionic-backed, huge plugin ecosystem.

This shifts the risk calculus. Tauri 2 mobile is no longer "betting on a young framework" — it's a real production option.

---

2. Options

Five realistic paths. Every one keeps the Vite web build as the source of truth.

Option A — Capacitor (mobile) + Electron (Steam) recommended

Two wrappers, one web build.

Mobile — Capacitor. Drops the web build into native Android/iOS shells. WebView-based (WKWebView on iOS, Chromium System WebView on Android). Phaser's WebGL runs as-is. Plugin ecosystem covers everything we'll need: in-app purchases (`@capacitor-community/in-app-purchases` or RevenueCat), haptics, status bar, preferences, analytics, deep links, share sheet.

Desktop / Steam — Electron. Chromium + Node, ~100 MB binary. Mature Steam integration via `steamworks.js` — achievements, cloud saves, overlay, rich presence. `electron-builder` produces Windows `.exe` + macOS `.dmg` + Linux `AppImage`.

Pros

- Most mature toolchain, battle-tested across all three platforms.
- `steamworks.js` is the de-facto Steam integration — examples, docs, shipping games.
- Plugin ecosystems cover every native surface we'd need.
- Can ship PWA + native builds in parallel.

Cons

- Two toolchains, two build pipelines.
- Electron binaries are ~100 MB (not a Steam concern; players are used to it).

Option B — Capacitor (mobile) + Tauri (Steam)

Same mobile story, lighter desktop.

Desktop — Tauri. Uses the OS's system webview (WebView2 on Windows, WKWebView on macOS, WebKitGTK

on Linux). Binaries ~10–20 MB. Rust backend.

Pros

- 5–10× smaller desktop downloads than Electron.
- Lower memory + CPU overhead at runtime.
- Actively developed, modern architecture.

Cons

- **Steam integration is less mature.** Integrating Steamworks beyond the basics requires writing Rust — `tauri-plugin-hal-steamworks` and `steamworks-rs` exist but have far fewer shipping examples than Electron's `steamworks.js`. Steam Overlay has known open issues on Tauri.
- System webview inconsistency: WebGL / Phaser 4 behavior varies slightly across WebView2 / WKWebView / WebKitGTK. Needs testing on each.
- Requires Rust toolchain in the build pipeline.

Option C — Tauri 2 unified (all three platforms)

Single framework, single build config, covers Android + iOS + desktop.

Pros

- Smallest binaries across the board.
- One toolchain, one mental model.
- Consistent plugin API across platforms.

Cons

- **Steam integration weakness from Option B still applies.** It's the weakest link for this project.
- Mobile is newer than Capacitor — plugin ecosystem thinner, especially for IAP. You'll either write your own native plugin or use a less-proven community one.
- Webview inconsistency on mobile too.

Worth re-evaluating in 12 months once more Steam games ship on Tauri. For a launch now, the Steam story is the blocker.

Option D — Capacitor everywhere (Android / iOS / Desktop via Electron adapter)

Capacitor has an Electron adapter that wraps the same Capacitor project as a desktop app via Electron under the hood.

Pros

- Single Capacitor project structure for three platforms.
- Shared plugin API across stores (one "buy coins" flow that works everywhere).

Cons

- The Electron adapter is thinner than standalone Electron — less flexibility for Steam-specific features (overlay, achievements, rich presence).
- Most projects end up ejecting to standalone Electron anyway once Steam integration lands.

Worth considering if cross-platform IAP unification is a top priority. Otherwise Option A is cleaner.

Option E — PWA + TWA (Android) + Safari home-screen (iOS) + no Steam

Lowest effort:

- **Android:** Trusted Web Activity wrapping the PWA as a Play Store app.
- **iOS:** Safari "Add to Home Screen" — cannot ship to App Store.
- **Steam:** not applicable.

Rejected — misses iOS App Store and Steam. Worth keeping the PWA track alive *in parallel* for web users, but not the answer to cross-platform distribution.

---

3. Comparison

Approach	Android	iOS	Steam	Toolchains	Desktop binary	Mobile maturity	Steam maturity
A. Capacitor + Electron				2	~100 MB	High	High
B. Capacitor + Tauri			⚠	2	~15 MB	High	Medium
C. Tauri 2 unified			⚠	1	~15 MB	Medium	Medium
D. Capacitor all-in-one			⚠	1	~100 MB	High	Low–Medium
E. PWA + TWA				0.5	N/A	High	N/A

---

4. Recommendation: Option A (with eye on B for v2)

Capacitor for mobile + Electron for Steam. Reasons:

1. **Steam is the weakest link across the board, and Electron is where Steam integration is easiest.** `steamworks.js` is mature, documented, and used by real shipping games. Tauri's Steam story is improving but still requires Rust for anything non-trivial.
2. **Capacitor's mobile plugin ecosystem is deeper than Tauri 2's.** IAP, analytics, deep links, push — all documented, all community-supported.
3. **The 100 MB Electron binary is not a real constraint on Steam.** Players download 50 GB games regularly. It's a constraint on free mobile-only indie apps, but we're shipping native builds for all three stores.
4. **Lowest risk of a late surprise.** Every native surface we'd touch has existing Capacitor/Electron plugins we can install.

Keep Option B on the watchlist. If Steam becomes less important than we think, or if Tauri's `steamworks-rs` ecosystem matures significantly during development, swapping Electron → Tauri is a 1–2 week detour since both wrap the same web build.

---

5. Shared prep (applies to every option)

Before wrapping, the web app needs cleanup that benefits every approach. Could land on `develop` now, independent of which wrapper we pick.

1. **Relative asset paths.** `vite.config.ts` currently has `base: '/tower_defence/'` for GitHub Pages. Native shells want `base: './'` so assets resolve relative to the bundle. Keep both configs, switch by env.
2. **Asset bundling review.** Big spritesheets currently loaded async from `public/`. Fine over HTTP, fine in a packaged app too, but confirm the file count stays reasonable for mobile app-bundle limits (Play Store cap is 200 MB for the base APK, with additional asset packs after that).
3. **localStorage → Capacitor Preferences.** localStorage still works in all wrappers, but Capacitor Preferences is more reliable — WKWebView on iOS can drop localStorage under memory pressure.
4. **Clipboard multiplayer flow.** Manual SDP exchange uses `navigator.clipboard`. Native needs permission config. Consider an in-app share-sheet alternative.
5. **Safe areas.** iOS notch + Android nav bar. Add CSS `env(safe-area-inset-*)` to sidebar + tower dock.

6. **Android back button.** Decide behavior per screen: exit? pause? back through menus?
7. **Orientation lock.** Decide per-mode. Tutorial is landscape-friendly; main UI works both.
8. **Audio unlock.** Already in Phaser — verify first-tap unlock fires cleanly on mobile wrappers.
9. **Monetization refactor.** Current `StorePersistence` is local-only. Real IAP needs: plugin integration, server-side receipt validation (optional but best-practice), product IDs configured per store.

---

6. Rollout (Option A)

Phase 0 — shared prep (1–2 days)

Items 1–8 above. Independent of wrapper. Can land on develop before any native work starts.

Phase 1 — Capacitor Android (2–3 days)

- `npm install @capacitor/core @capacitor/android @capacitor/cli`
- `npx cap init && npx cap add android`
- Build → sync → open in Android Studio → run on emulator / real device.
- Icon + splash assets via `capacitor-assets`.
- Play Console internal-testing track upload.

Phase 2 — Capacitor iOS (2–3 days, **requires a Mac**)

- `npx cap add ios`
- Xcode project setup, signing certs, provisioning profiles.
- TestFlight submission.
- **Prereq:** Apple Developer account ($99/yr), physical Mac or rented cloud Mac (MacStadium / AWS).

Phase 3 — Electron for desktop (2–3 days)

- `npm install --save-dev electron electron-builder`
- Main-process file that loads the Vite build.
- `electron-builder` config for Win / Mac / Linux.
- Smoke test on each OS.

Phase 4 — Steam (1–2 days + admin overhead)

- Steamworks partner account (one-time $100 per title).
- `steamworks.js` integration for achievements + cloud saves + overlay.
- SteamPipe upload config.
- Steam store page assets (capsule art, screenshots, trailer, description).

Phase 5 — IAP + monetization (scope-dependent)

- Capacitor IAP plugins on mobile (Google Play Billing, Apple StoreKit).
- Steam microtransactions / DLC if applicable.
- Server-side receipt validation.

---

7. Open questions to lock in before coding

1. **Monetization model** — free-to-play with IAPs, or paid upfront? Shapes how much store plumbing we need at launch.
2. **Mac access** — do we have a Mac? Required for iOS builds and notarised macOS desktop builds.
3. **Multiplayer flow on native** — keep manual-SDP-clipboard or invest in a signalling server?

4. **Launch order** — Android first to iterate fast, then iOS + Steam? All three simultaneously (more work, tighter message)?
5. **CI infra** — three-platform builds need different runners (Linux for Android, Mac for iOS, any for Electron). GitHub Actions can do all three but burns minutes.

---

8. Sources

- Tauri 2.0 Stable Release announcement
- Tauri Mobile overview (2026)
- Tauri vs Electron (2026 benchmark)
- Tauri vs Capacitor comparison (2026)
- Steam Overlay + Tauri open issue
- `tauri-plugin-hal-steamworks` on crates.io
- Capacitor official site
- Porting a browser-based game to Steam (Schemescape writeup)